

STAT 432 - Final Report: Churn Prediction

Jada Giddens (jadag2), Colin Gu (clgu2), Adi Agarwal (aa123), Dev Langaliya (devl2)

2026-04-22

Abstract:

Predicting customer churn is very important for business success, as understanding and minimizing churn directly impacts profitability and growth. This project develops and compares multiple classification models to predict customer churn, including logistic regression, K-nearest neighbors (KNN), support vector machines (SVM), and ensemble methods (bagging and random forest). Given the class imbalance presented in the dataset (86% didn't churn, 16% did), we used sensitivity and test error as our key model evaluation metrics to ensure they could effectively identify at-risk customers. Our results demonstrate that ensemble methods outperformed single-model approaches, with random forest achieving the lowest test error and highest sensitivity. Additionally, logistic regression provided valuable interpretability, as it offered clear insights into which predictor variables most strongly influenced churn. Our findings can help businesses develop data-driven strategies to target customers at risk of churning.

Contributions:

Adi worked on introduction/abstract/interpretations, literature review, and the KNN model. Jada worked on introduction/abstract/interpretations, literature review, and the SVM model. Colin worked on introduction/abstract/interpretations, literature review, and the Logistic regression model. Dev worked on introduction/abstract/interpretations, literature review and the decision tree model.

Introduction

In this project, our goal is to predict customer churn in an e-commerce context using a dataset containing 5,630 customer records with 19 features. The dataset was provided by Ankit Verma on Kaggle and includes information on customer demographics, behavioral metrics (such as app usage frequency and purchase activity), satisfaction indicators (ratings and complaints), and engagement patterns. Our target variable is whether a customer churned (stopped using the service). By building and comparing multiple classification models, we aim to identify the most effective approach for predicting churn, enabling the business to take targeted retention actions such as improving service quality or offering targeted incentives to at-risk customers.

Churn is a common problem businesses face, so there is quite a bit of literature already about the best methods to predict it. According to Prabadevi et al. (2023), methods like stochastic gradient boosting, random forest, KNN (K Nearest Neighbors), and logistic regression can be used to predict customer churn. In their study comparing these methods, they found that stochastic gradient boosting performed best with 84% accuracy while KNN performed the worst. Similarly, The Pecan Team (2026) reports that ensemble methods like random forest and boosting provide more accurate models but are harder to interpret.

For this project, we decided to explore using multiple classification models: KNN, logistic regression, decision trees, bagging, random forest, and support vector machines. Logistic regression serves as our baseline model, providing easily interpretable results that help us understand the effect of the individual variables on churn. Decision trees also function as a baseline, allowing us to compare single-tree performance against ensemble methods. Bagging and random forest are ensemble techniques that combine multiple decision trees to improve accuracy and reduce overfitting. KNN is a non-parametric algorithm that classifies data points based on the majority class of their k nearest neighbors, offering a simple yet effective approach. Finally, support vector machines find the optimal hyperplane to maximize the margin between classes, which will be useful in any handling of non-linear relationships present.

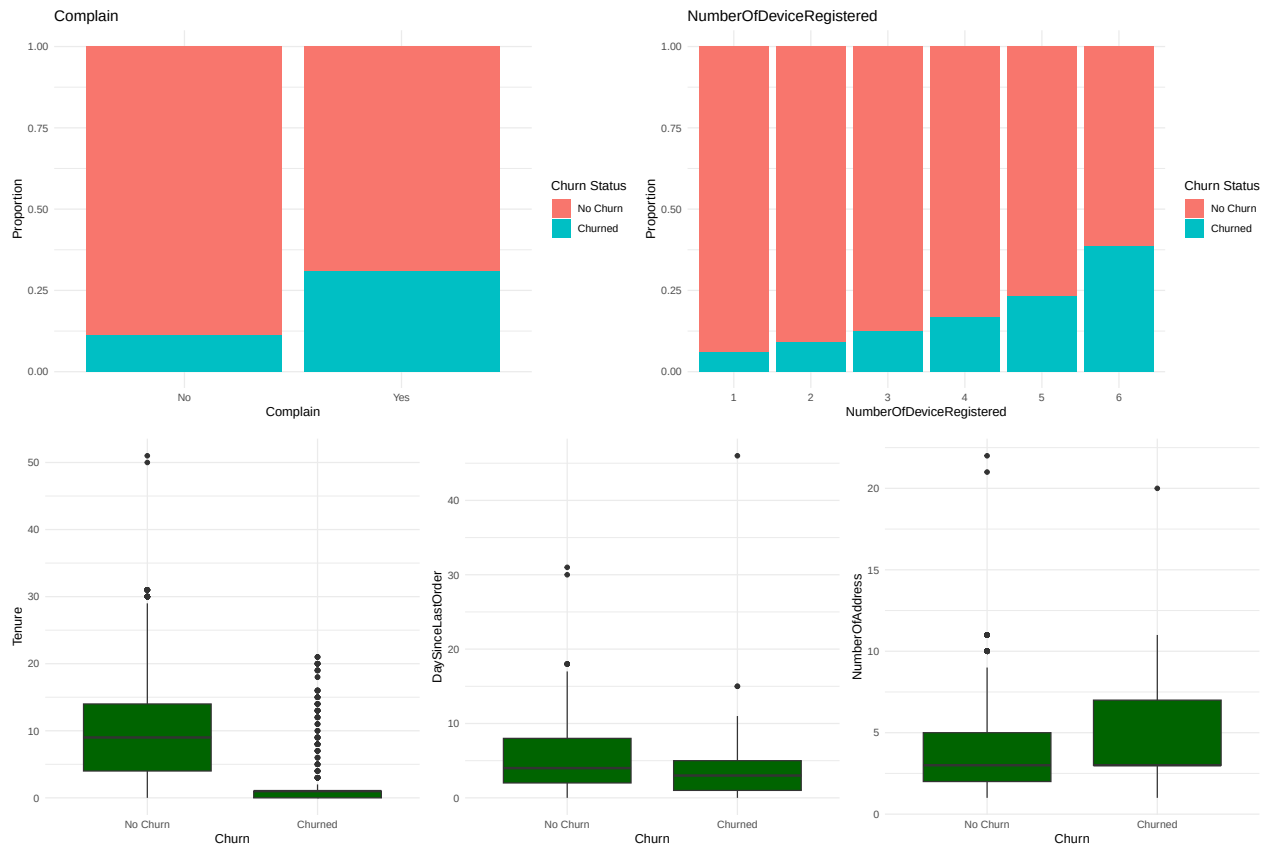
Exploratory Data Analysis

Before analyzing the data, we began with a cleaning process. The original dataset contained 5,630 rows and 19 columns. Most variables had no missing values; however, the columns `Tenure`, `HoursSpendOnApp`, `OrderAmountHikeFromLastYear`, `CouponUsed`, `OrderCount`, and `DaySinceLastOrder` each had less than 5% missing data. Given the relatively small proportion of missing values, we chose to remove all rows containing any missing entries. After this step, the cleaned dataset consisted of 3,774 rows and 19 columns.

During exploratory data analysis, we observed a class imbalance in the target variable `Churn`. Specifically, 83.2% of observations correspond to customers who did not churn (0), while only 16.7% represent customers who did churn (1). This imbalance will be important to consider when evaluating model performance, particularly accuracy.

In examining the distributions of the variables, we found that most numerical features are right-skewed, with `CashbackAmount` being the only exception. For categorical variables, the majority of users preferred debit cards as their payment method, were male, spent approximately 3 hours on the app, had four registered devices, primarily purchased laptops and accessories, reported a satisfaction score of 3, were married, had not filed complaints, and resided in Tier 1 cities.

To assess the relationships between churn and the predictor variables, we constructed proportion bar plots and box plots. From the proportion plots, customers who had filed complaints and those with six registered devices showed higher churn frequencies. The box plots indicated that customers with lower tenure, fewer days since their last order and a higher number of registered addresses were more likely to churn. These variables may serve as important predictors in the model.



Model 1: Logistic Regression

For Logistic Regression we decided to do it with 2 approaches. Logistic regression using Best Subset Selection based on BIC and Logistic Regression using LASSO. Before performing our logistic regression we split the data into train and test data based on an 80:20 ratio (80% train and 20% test), and encoded the categorical variables using one-hot encoding.

From using Best Subset Selection that minimizes BIC we found that it ended up selecting 12 predictors being: Tenure, CityTier, WarehouseToHome, NumberOfDeviceRegistered, SatisfactionScore, NumberOfAddress, Complain, OrderCount, DaySinceLastOrder, PreferredLoginDeviceMobile Phone, PreferredOrderCat, and MaritalStatusSingle.

```
##
## Call:
## glm(formula = Churn ~ Tenure + 'PreferredLoginDeviceMobile Phone' +
##      CityTier + WarehouseToHome + NumberOfDeviceRegistered + 'PreferredOrderCatLaptop & Accessory' +
##      SatisfactionScore + MaritalStatusSingle + NumberOfAddress +
##      Complain + OrderCount + DaySinceLastOrder, family = binomial,
##      data = train_encoded)
##
## Coefficients:
##
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -4.94630    0.41584 -11.895 < 2e-16 ***
## Tenure           -0.22103    0.01411 -15.668 < 2e-16 ***
## 'PreferredLoginDeviceMobile Phone' -0.53631    0.13453  -3.987 6.70e-05 ***
## CityTier          0.43176    0.07033   6.139 8.32e-10 ***
## WarehouseToHome   0.04753    0.00703   6.761 1.37e-11 ***
## NumberOfDeviceRegistered 0.32698    0.06605   4.950 7.42e-07 ***
```

```
## 'PreferredOrderCatLaptop & Accessory' -1.25849    0.14385   -8.749   < 2e-16 ***
## SatisfactionScore          0.25821    0.04692    5.504  3.72e-08 ***
## MaritalStatusSingle        0.95764    0.12820    7.470  8.02e-14 ***
## NumberOfAddress            0.20676    0.02420    8.545   < 2e-16 ***
## Complain                   1.65209    0.13260   12.459   < 2e-16 ***
## OrderCount                 0.18780    0.03244    5.788  7.11e-09 ***
## DaySinceLastOrder         -0.12896    0.02604   -4.953  7.31e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 2713.5  on 3018  degrees of freedom
## Residual deviance: 1681.1  on 3006  degrees of freedom
## AIC: 1707.1
##
## Number of Fisher Scoring iterations: 6
```

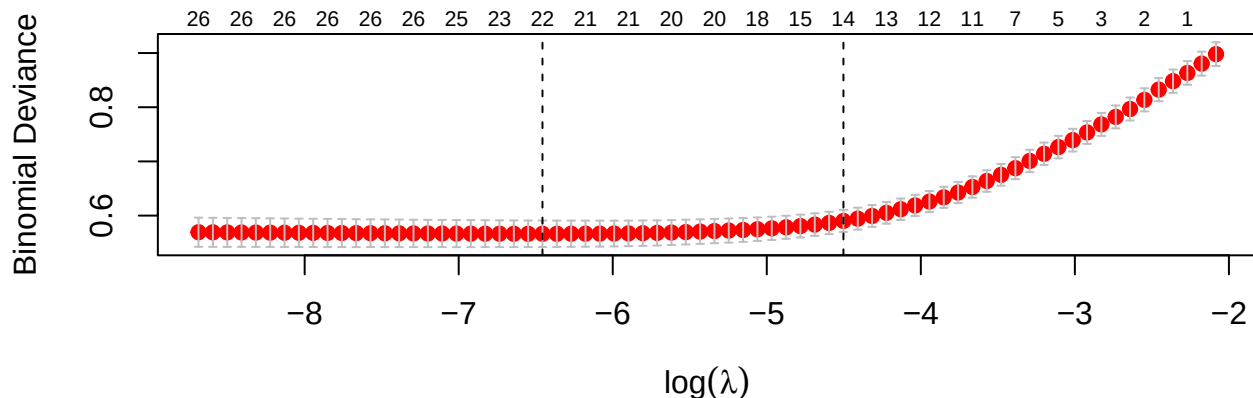
After running logistic regression on those 12 predictors and looking at the results we can see that all coefficients are statistically significant at a 0.05 level. Comparing the Null deviance with our Residual Deviance we can see a significant drop from 2713.5 down to 1681.1. Meaning that our model explains a lot of the variation in churn.

Looking at the predictors we can see customers who filed complaints, are single, are in higher city tiers, who have multiple addresses, have more devices registered, are further from the warehouses, placed more orders, and surprisingly customers who gave higher satisfaction scores are more likely to churn (perhaps people who are satisfied with the product simply don't feel the need to churn as they are content with what they have. For example if someone's phone worked perfectly for the rest of my life, a company like Samsung may never sell another phone to them as they don't feel like they need another one).

Meanwhile people who have tenured for longer, who primarily order/buy laptops and accessories, who order via mobile phone, and who have ordered more recently are less likely to churn.

Now to analyse logistic regression with LASSO, first what we had done was find the hyper parameter for lambda. We did this by using 10 fold cross validation and chose to use lambda min (0.001570901). Then using lambda min we fitted a model and found out that there were 22 predictors remaining (Below on the LASSO CV plot we can see that at lambda min there are 22 predictors. Also note that categorical predictors were split up into different levels). After that we did logistic regression on the 22 predictors:

LASSO Cross-Validation



```
##                                     Estimate Std.Error  z.value  p.value
```

```
## (Intercept) -3.7490 0.5393 -6.9509 3.629e-12
## Tenure -0.2397 0.0156 -15.4104 1.394e-53
## 'PreferredLoginDeviceMobile Phone' -0.5597 0.1363 -4.1060 4.026e-05
## CityTier 0.2948 0.0816 3.6127 3.030e-04
## WarehouseToHome 0.0479 0.0072 6.6236 3.506e-11
## NumberOfDeviceRegistered 0.3747 0.0694 5.3969 6.779e-08
## 'PreferedOrderCatLaptop & Accessory' -1.7603 0.2085 -8.4443 3.058e-17
## 'PreferedOrderCatMobile Phone' -0.7286 0.2214 -3.2914 9.971e-04
## PreferedOrderCatOthers 2.2460 0.8797 2.5532 1.067e-02
## SatisfactionScore 0.2490 0.0476 5.2273 1.720e-07
## MaritalStatusMarried -0.4019 0.1906 -2.1089 3.495e-02
## MaritalStatusSingle 0.7001 0.1908 3.6689 2.436e-04
## NumberOfAddress 0.2123 0.0246 8.6344 5.901e-18
## Complain 1.6513 0.1339 12.3323 6.067e-35
## OrderCount 0.2143 0.0442 4.8456 1.262e-06
## DaySinceLastOrder -0.1508 0.0268 -5.6161 1.953e-08
```

```
##
```

```
## Null deviance: 2713.503 on 3018 degrees of freedom
```

```
## Residual deviance: 1650.349 on 2996 degrees of freedom
```

```
## AIC: 1696.349
```

Since not all of the predictors were significant, I cut out the output to only show the ones that were significant. Looking at these results we can see that the Null deviance is the same as before at 2713.5 and now the Residual deviance is a bit lower at 1650.3. Which means that it explains more of the variance in churn, but we do have to note that it includes more predictors than the other model.

Analyzing only the significant predictors we can see that people who are of higher city tier, live farther away from a warehouse, prefer paying with an E wallet, have more devices registered, who like ordering 'other' goods (this has a wide standard error so it may not be the most trustworthy), people who are more satisfied with the product, people who are single, people who have multiple addresses, people who complain about the product, and people who order a lot of products are more likely to churn.

Meanwhile, longer tenured customers, people who use a mobile phone to order, like ordering laptops/accessories or mobile phones, people who are married, and people who have ordered more recently are less likely to churn.

Model	Test_Error	Sensitivity	Specificity
Logistic (BIC)	0.1060	0.5846	0.9584
Logistic (LASSO)	0.1113	0.5769	0.9536

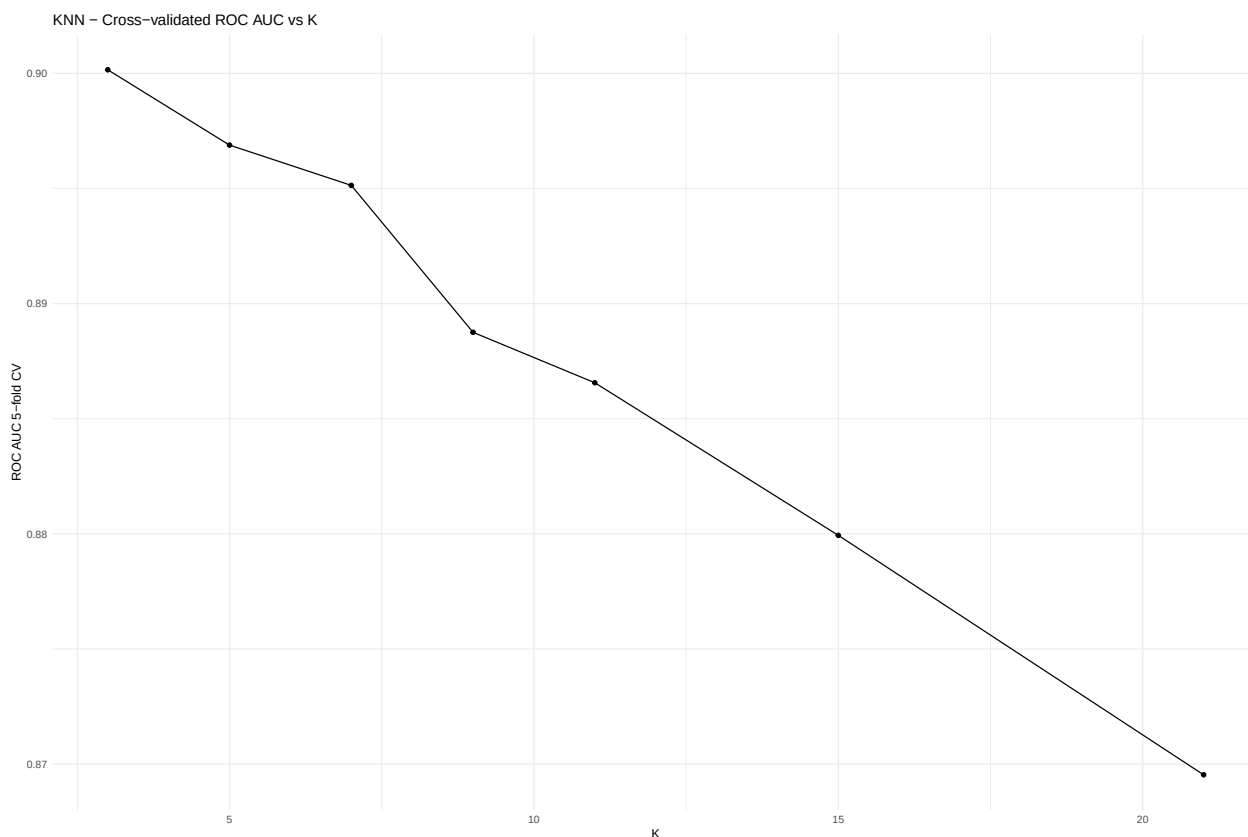
Looking at the confusion matrix for the best subset selection (top matrix) and the confusion matrix for the LASSO logistic regression (bottom) we can calculate test error, sensitivity and specificity. After calculating those values we can see that the best subset model using BIC performs better as it has a lower test error (.1060 vs 0.1113), higher sensitivity (0.5846 vs 0.5769) (also this is somewhat low, meaning that both models seem to have trouble identifying churners. This is likely because of the class imbalance our dataset has), and higher specificity (0.9584 vs 0.9536) than the LASSO logistic regression model, but ultimately the difference in test error, sensitivity and specificity is not huge. Comparing both the best subset selection with BIC logistic regression and LASSO logistic regression, we can see that both of them are similar in terms of the predictors that they find the most important with all of the predictors selected with best subset selection

being used in LASSO logistic regression and also being considered significant predictors. We can see that LASSO regression takes into account more predictors even though a lot of them aren't significant. Overall both logistic regression models have similar results and can both be considered when comparing with the other models (KNN, Decision Trees, etc.)

Model 2: KNN

For KNN, we performed two key steps: preprocessing and hyperparameter tuning. During preprocessing, we converted all categorical variables to numerical values, split the data 80/20 for training and testing, and scaled all features to a consistent range using only the training data to prevent data leakage. For tuning, we employed 5-fold cross-validation optimizing on ROC AUC, since accuracy alone can be misleading with imbalanced datasets. We tested K values of 3, 5, 7, 9, 11, 15, and 21, with cross-validation selecting K=3 as optimal (ROC AUC = 0.918). This means the model classifies each customer based on the 3 most similar customers in the training data.

The KNN model with K=3 achieved 91.6% accuracy and a Kappa of 0.67, indicating strong performance well above random guessing. More relevant to the churn prediction problem are Precision (81.8%) and Recall (64.3%). The 81.8% precision means that when the model flags a customer as likely to churn, it is correct approximately 82% of the time—enabling targeted, cost-efficient retention campaigns. However, the 64.3% recall reveals a significant limitation: the model misses roughly 36% of actual churners, meaning a substantial portion of at-risk customers would not receive intervention. Unlike logistic regression, KNN provides no coefficient-level insight into why customers churn (e.g., the impact of complaints or relationship status); instead, it offers a solid predictive signal based purely on customer similarity.



Metric	Value
Accuracy	0.8846
Precision	0.6757
Recall	0.5952

Metric	Value
F1 Score	0.6329
Kappa	0.5648

Model 3: Support Vector Machine

For our third modeling approach, we used support vector machines (SVM), a method commonly applied to binary classification problems such as customer churn. SVMs perform well with high-dimensional data and can capture non-linear relationships through the use of kernel functions. We evaluated the three kernel types used in class: linear, polynomial, and radial. The linear kernel was used as a baseline under the assumption that the relationship between churn and the predictor variables was linear. The polynomial and radial kernels were selected to account for potential non-linear relationships. To address the class imbalance, we also implemented a weighted radial model that penalized misclassification of churners more heavily by assigning class weights of 1:2, 1:3, 1:5, and 1:10.

Before fitting the models, the data was split into training and testing sets using an 80/20 split. Nominal categorical variables were transformed using one-hot encoding, as SVM requires numerical inputs, and the ordinal variable satisfaction score was left numeric.

All features were standardized using z-score normalization (mean = 0, standard deviation = 1). This step was essential because SVM is a distance based algorithm that depends on the relative position of observations to the decision boundary. Without scaling, variables with larger magnitudes could disproportionately influence the model.

All variables were included in each model. To tune the hyperparameters the tuning function was used using 8-fold cross-validation on the training data for each kernel type. The linear kernel was tuned on cost (c(0.001, 0.01, 0.1, 1, 5, 10, 100)), the polynomial kernel was tuned on cost and degree (degree = 2, 3), and the radial kernel was tuned on cost and gamma (c(0.001, 0.01, 0.1, 1)). The weighted radial model was tuned across all cost and gamma combinations for each class weight ratio. The values used to tune were chosen based on standard practice. Code for each model is provided in Appendix C.

The best-performing model from each kernel type was selected based on cross-validation performance and evaluated on the test dataset. The results are shown in the table below.

Kernel	Accuracy	Test_Error	Sensitivity	Specificity	Cost
Linear	0.90	0.10	0.55	0.97	0.1
Polynomial	0.94	0.06	0.86	0.96	100.0
Radial	0.96	0.04	0.79	0.99	10.0
Radial_Weighted	0.95	0.05	0.78	0.99	5.0

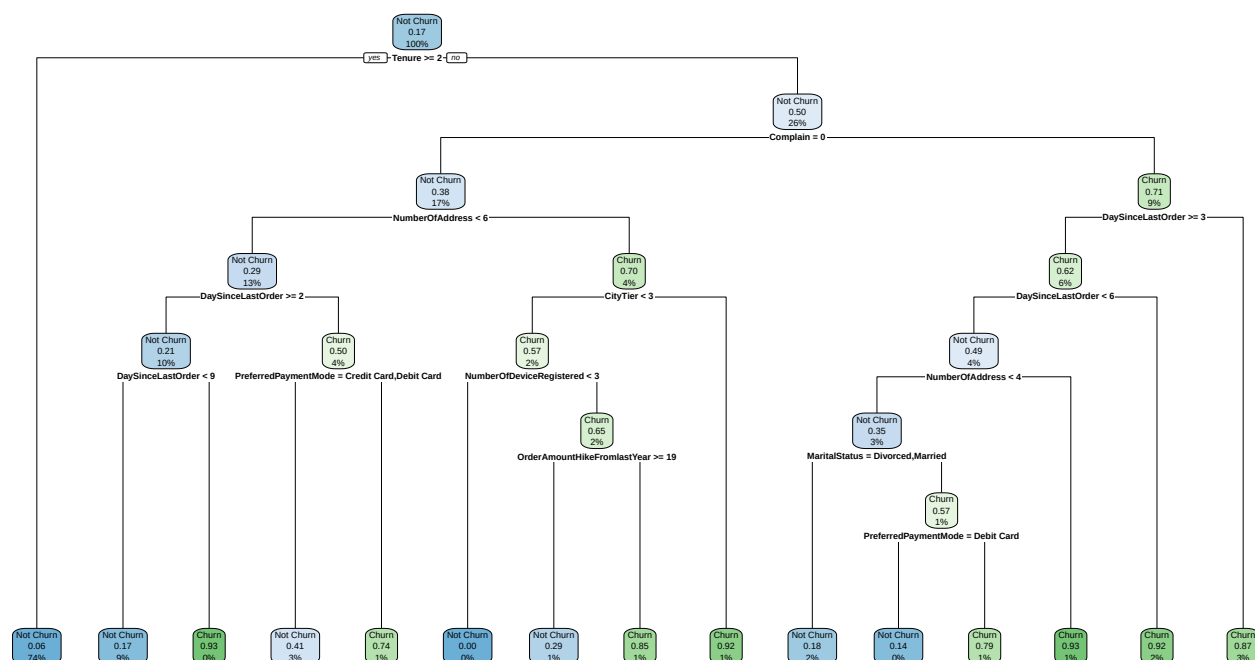
The radial kernel achieved the highest overall accuracy, while the polynomial kernel produced the highest sensitivity, meaning it was better at correctly identifying customers who churn. The linear model performed the worst, most likely because the relationships between the predictor variables and churn were not linearly separable.

It's important to note that the polynomial model also had the highest cost value at 100, meaning it had the narrowest margin with less tolerance for classification errors. This suggests that the model is fitting more closely to the training data, potentially increasing variance and overfitting. This trade-off between sensitivity and generalization should be carefully considered if used in a business setting.

From a business perspective, the polynomial kernel is preferred despite its slightly lower accuracy, because it effectively identifies customers at risk of churning. A limitation is that this model suffers from a lack of interpretability, as there is no clear way to determine which variables contributed most to the churn predictions. This makes it difficult for businesses to take actionable steps based on the model's results and may limit its practical implementation.

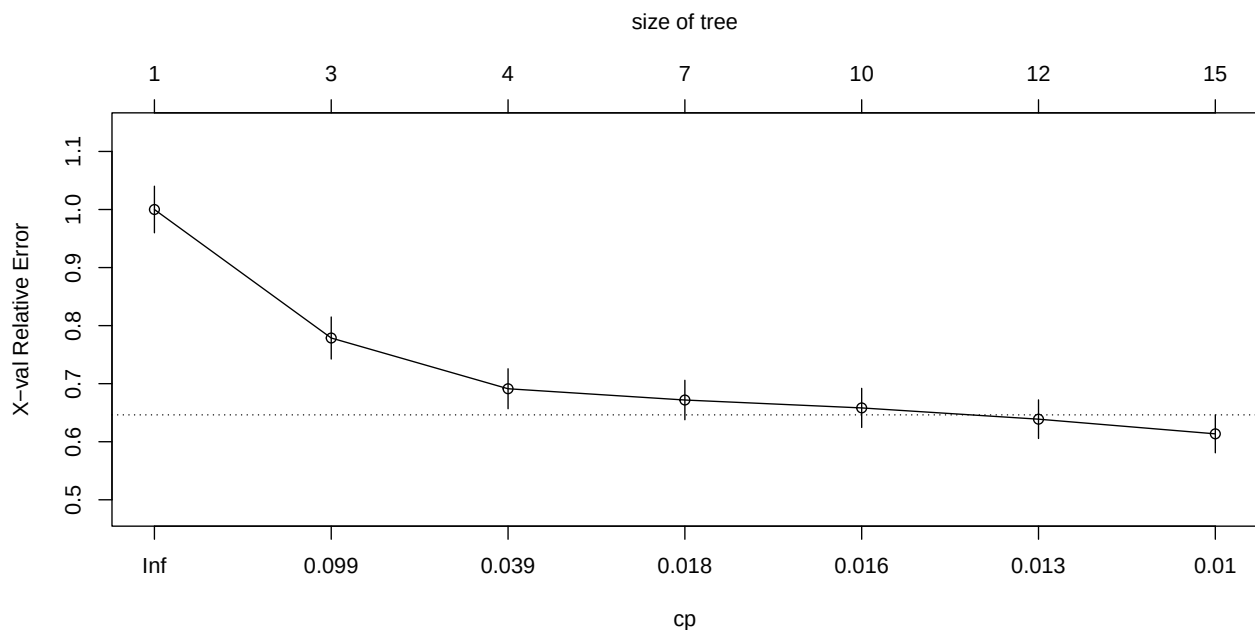
Model 4: Decision Trees

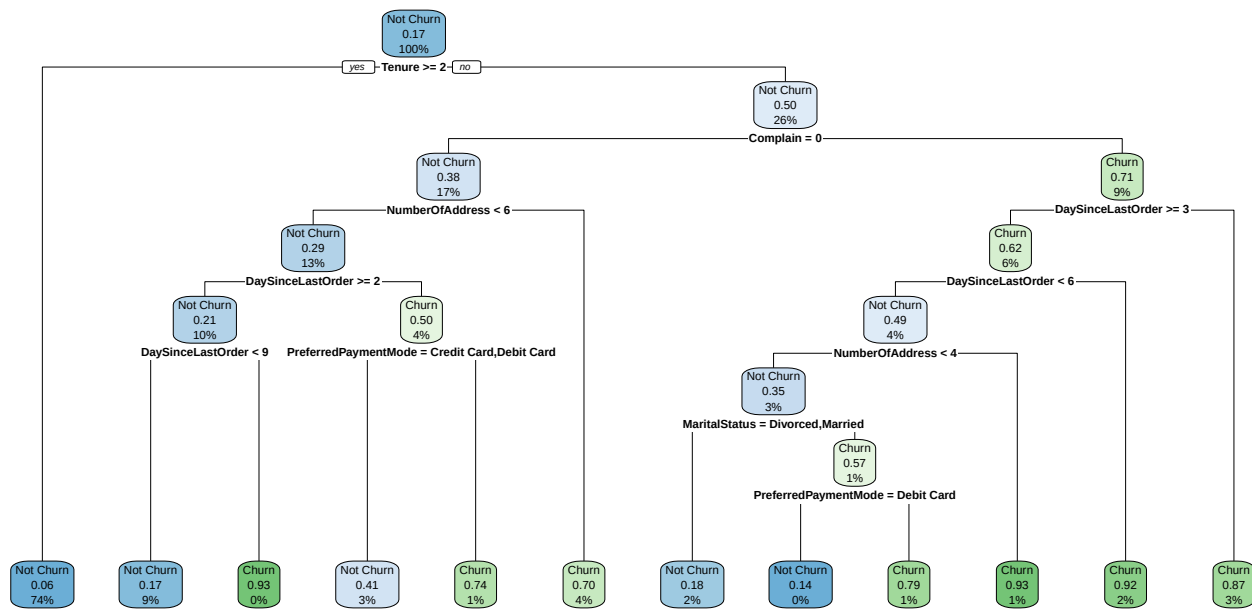
To begin, we split the data into an 80-20 train test split, and then fit a single classification tree on the training set.



The initial classification tree achieves a test error of about 0.0808, and we can see that this tree is massive with several branches and terminal nodes.

To avoid the potential danger of overfitting, we proceed to prune the tree. When a tree is too large, it has low bias and high variance, meaning small changes to our training set can really affect the tree. We use cost complexity pruning to penalize this excess in tree size, as the penalty directly increases with the amount of terminal nodes in a tree. Then, we use the 1-SE rule to find the best cp value that maximizes the cv error within one standard deviation of the minimum error.

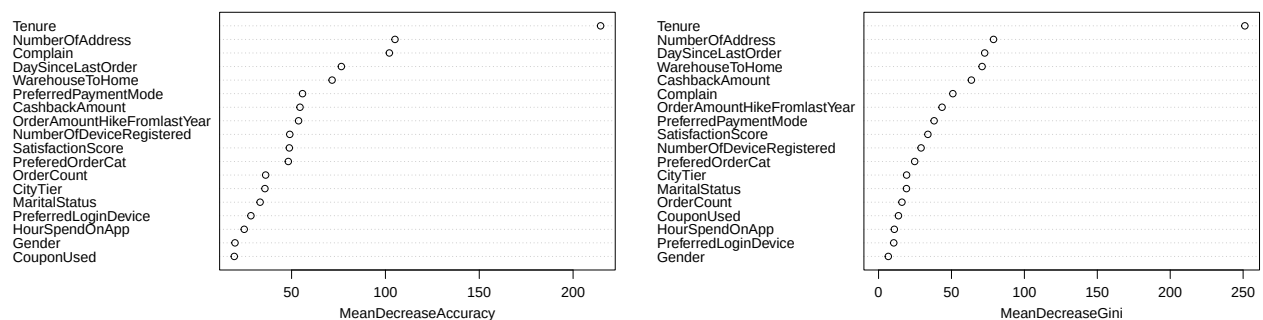




Using the 1-SE rule, we choose the tree with a cp value of 0.01100324, resulting in 12 terminal nodes. This pruned tree gives us a test error of 0.0847682, which is slightly higher compared to the unpruned tree, indicating that pruning does not help in constructing a better tree in this scenario.

Another approach we can use to improve our models is bagging, where we can reduce the variance of a tree by fitting B trees on bootstrapped samples of the training data and combining. We set mtry to the amount of predictors (18), indicating each predictor is taken into consideration at every split.

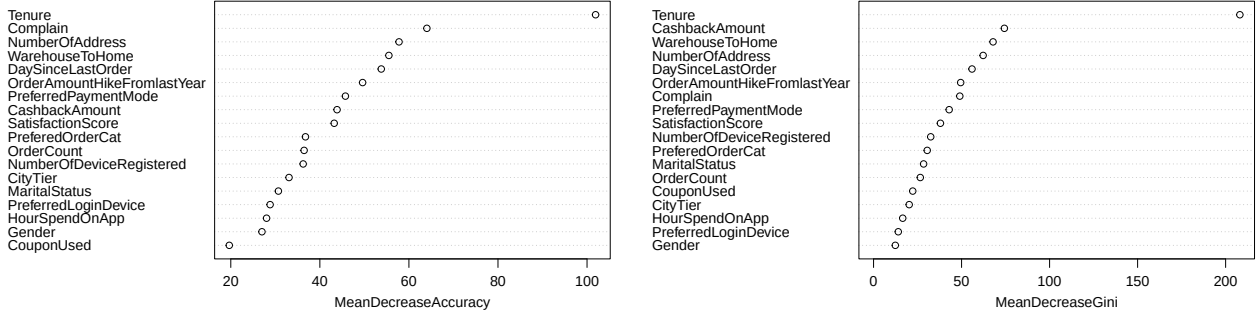
Variable Importance: Bagging



The test error achieved by bagging is 0.02119205, much better than the previous two approaches, and aligning with the idea that fitting several trees reduces variance. Looking at the variable importance plot, we can see that variables like Tenure hold significant influence within the model, with Complain and DaySinceLastOrder also holding some influence in terms of MeanDecreaseAccuracy.

Unfortunately, bagging has the issue such that one super influential variable will dominate splits across all B trees (in our case Tenure), causing correlation across trees. This is where Random Forest comes in, as it only allows a subset of the predictors (the floor of the square root of the amount of predictors is the size of this subset) to be taken into consideration at each split.

Variable Importance: Random Forest



The non-tuned random forest achieves a test error of 0.01456954, the best yet. We can also see in the variable importance plot that for MeanDecreaseAccuracy, Tenure is no longer overwhelmingly more influential than the rest of the predictors.

To see if we can further optimism the Random Forest model, we tune the mtry and nodesize parameters. We fit a random forest model for each combo given our grid dimensions, and determine which combination achieves the lowest OOB error rate.

Our tuning tells us that having a nodenize of 3 and mtry of 7 is optimal. This resulted a test error rate of 0.02119205, which actually hurts the accuracy relative to the untuned random forest model.

The table below highlights the models fit and their overall test error.

Model	Test_Error	Sensitivity	Specificity
Simple Classification Tree	0.0808	0.5776	0.9812
Pruned Classification Tree	0.0848	0.5776	0.9765
Bagging	0.0212	0.9397	0.9859
Random Forest	0.0146	0.9138	0.9984
Tuned Random Forest	0.0212	0.8793	0.9969

As expected, the single classification tree has the second highest test error in terms of being able to correctly identify churning customers, and we sacrifice a bit more test error for a tree that is easier to interpret in the pruned classification tree. Then with bagging, we significantly reduce variance via fitting 500 trees, and random forest takes this even further by restricting the amount of parameters the model is allowed to split from to ensure one predictor isn't taking over the fitting process, which is why the plain random forest model has the lowest test error of 0.01456954. Unfortunately tuning the parameters for random forest did not yield in a better tree.

Conclusion

Our aim for this project was to identify the most effective approach for predicting churn to enable the e-commerce business to take targeted actions to improve their service and reduce customer loss. We compared several classification models including logistic regression, KNN, decision trees, ensemble methods (bagging and random forest), and support vector machines. Using sensitivity (the true positive rate for identifying customers who will churn), Random Forest without tuning performed best with a sensitivity of 91.38%. The tree-based structure of this model allows businesses to identify specific subsets of customers at highest risk of churning and tailor retention interventions accordingly.

While Random Forest achieved the best overall performance, logistic regression provided clearer, coefficient-based insights into which customer characteristics most strongly influence churn (complaints, marital status, tenure). Similarly, the single decision tree offered interpretability at the cost of lower accuracy (test error of 0.0808). The class imbalance in our dataset (83.2% non-churners vs. 16.8% churners) presented a consistent challenge across models, so future work could take this into account more when splitting the data for test and train.

Appendix A: Logistic Regression

This is the code using regsubsets to preform best subset selection and find the best predictors. Then those predictors are fitted into a logistic regression model. I used Claude to generate this code, I compared the code to what we did in the homework and the code looks correct and makes sense.

```
subset_model <- regsubsets(Churn ~ ., data = train, nvmax = ncol(train) - 1,
                          method = "exhaustive")

subset_summary <- summary(subset_model)

best_n <- which.min(subset_summary$bic)
best_vars <- names(coef(subset_model, best_n))[-1]

best_n
best_vars

model <- glm(Churn ~ Tenure + `PreferredLoginDeviceMobile Phone` + CityTier +
WarehouseToHome + NumberOfDeviceRegistered +
`PreferredOrderCatLaptop & Accessory`
+ SatisfactionScore + MaritalStatusSingle + NumberOfAddress +
Complain + OrderCount + DaySinceLastOrder,
data = train_encoded, family = binomial)

summary(model)
```

This is the code for finding the lambda min using 10 fold cross validation. After that LASSO is preformed using lambda min, and the variables selected are then fit into the logistic regression model. I used Claude to help me write the predictors to fit the model, all of the code here makes sense and matches what we did in class.

```
cv_lasso <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 1, nfolds = 10)

lasso_model <- glmnet(x_train, y_train, family = "binomial", alpha = 1,
                     lambda = cv_lasso$lambda.min)

logistic_lasso_model <- glm(Churn ~ Tenure + `PreferredLoginDeviceMobile Phone`
+ CityTier + WarehouseToHome + `PreferredPaymentModeCredit Card`
+ `PreferredPaymentModeDebit Card` + `PreferredPaymentModeE wallet`
+ PreferredPaymentModeUPI + GenderMale + NumberOfDeviceRegistered +
`PreferredOrderCatLaptop & Accessory` + PreferredOrderCatMobile +
`PreferredOrderCatMobile Phone` + PreferredOrderCatOthers + SatisfactionScore +
MaritalStatusMarried + MaritalStatusSingle + NumberOfAddress +
Complain + CouponUsed + OrderCount + DaySinceLastOrder,
data = train_encoded, family = binomial)
```

Appendix B: KNN KNN

This is the code for KNN, where it splits data, scales the features, and trains KNN model using 5 fold cross validation to pick the best K based on ROC. We then test model performance with generated predictions and confusion matrix and find variances to see which variables vary the most. I used Claude to help write the code to run the cross validation and evaluating on the unseen data. I cross checked with class notes and parts of ISLR and based on that the code makes sense to me, and I understand it all.

```

split_idx <- createDataPartition(df$Churn, p = 0.80, list = FALSE)
train= df[ split_idx, ]
test = df[-split_idx, ]
train_X= train %>% select(-Churn)
test_X= test %>% select(-Churn)
train_y =train$Churn
test_y= test$Churn
scaler=preProcess(train_X, method = c("center", "scale"))
train_X =predict(scaler, train_X)
test_X =predict(scaler, test_X)
set.seed(42)
ctrl= trainControl( method= "cv", number = 5, classProbs = TRUE,
                    summaryFunction = twoClassSummary)
levels(train_y) =c("No", "Yes")
levels(test_y)= c("No", "Yes")
knn_cv =train( x= train_X, y = train_y,method = "knn",
tuneGrid = data.frame(k = c(3, 5, 7, 9, 11, 15, 21)),
trControl = ctrl,metric = "ROC")
best_k= knn_cv$bestTune$k
ggplot(knn_cv)
+labs(title = "KNN - Cross-validated ROC AUC vs K",
      x = "K", y = "ROC AUC 5-fold CV") +theme_minimal()
pred_labels =predict(knn_cv, newdata = test_X)
pred_probs= predict(knn_cv, newdata = test_X, type = "prob")
cm = confusionMatrix(pred_labels, test_y, positive = "Yes")
feat_var= apply(train_X, 2, var)
feat_df = data.frame(Feature = names(feat_var), Variance = feat_var) %>%
arrange(desc(Variance))

```

Appendix C: SVM

```

costs = c(0.001, 0.01, 0.1, 1, 5, 10, 100)
gammas = c(0.001, 0.01, 0.1, 1)
degrees = c(2, 3)

# linear
svm_tune_linear = tune(svm, churn ~ ., data = train_scaled, kernel = "linear",
                      ranges = list(cost = costs),
                      tunecontrol = tune.control(cross = 8), scale = F)

# polynomial
svm_tune_poly = tune(svm, churn ~ ., data = train_scaled, kernel = "polynomial",
                    ranges = list(cost = costs, degree = c(2, 3)),
                    tunecontrol = tune.control(cross = 8), scale = F)

# radial
svm_tune_rad = tune(svm, churn ~ ., data = train_scaled, kernel = "radial",
                   ranges = list(cost = costs, gamma = gammas),
                   tunecontrol = tune.control(cross = 8), scale = F)

# radial weighted
# function to help with tuning
svm_weighted = function(x, y, cost, gamma, class_weight_ratio, ...) {
  svm(churn~., kernel = "radial", cost = cost, gamma = gamma,
      class.weights = c("0" = 1, "1" = class_weight_ratio), ...)
}

```

```

}

# use tune function to tune all hyper parameters
tune_result = tune(svm_weighted,
  churn ~ .,
  data = train_scaled,
  ranges = list(
    cost = c(0.001, 0.01, 0.1, 1, 5, 10, 100),
    gamma = c(0.001, 0.01, 0.1, 1),
    class_weight_ratio = c(2, 3, 5, 10)
  ),
  tunecontrol = tune.control(cross = 8),
  scale = FALSE)

# function to help evaluate each model
mod_metrics = function(mod) {
  ypred = predict(mod, test_scaled)
  tab = table(predicted = ypred, true = test_scaled$churn)
  accuracy = sum(diag(tab)) / sum(tab)
  test_error = 1 - accuracy
  sensitivity = tab["1","1"] / sum(tab[, "1"])
  specificity = tab["0","0"] / sum(tab[, "0"])
  return(c(
    Accuracy = accuracy,
    Test_Error = test_error,
    Sensitivity = sensitivity,
    Specificity = specificity
  ))
}

```

The code above shows the relevant steps I used to train the support vector machines. I used Claude to help tune the weighted radial model and to condense the code while ensuring it was implemented correctly. I believe the resulting model produced accurate and reasonable results, especially given the outcomes of the other models and the nature of the weighted radial algorithm

Appendix D: Decision trees

```

library(readxl)
library(dplyr)
library(rpart)
library(rpart.plot)

df <- read_excel("E Commerce Dataset.xlsx", sheet = "E Comm")
df <- df |>
  mutate(
    PreferredPaymentMode = case_when(
      PreferredPaymentMode == "CC" ~ "Credit Card",
      PreferredPaymentMode == "COD" ~ "Cash on Delivery",
      TRUE ~ PreferredPaymentMode
    ),
    PreferredLoginDevice = case_when(
      PreferredLoginDevice == "Phone" ~ "Mobile Phone",
      TRUE ~ PreferredLoginDevice
    )
  )

```

```

)
df <- df |> select(-CustomerID)
df_cleaned <- na.omit(df)
df_cleaned$Churn <- factor(df_cleaned$Churn, levels = c(0, 1),
                           labels = c("Not Churn", "Churn"))

ecom <- df_cleaned

library(rpart)
library(rpart.plot)
set.seed(7)

#split into train/test
ecom_id <- sample(1:nrow(ecom), floor(nrow(ecom) * .8))
ecom_train <- ecom[ ecom_id, ]
ecom_test <- ecom[-ecom_id, ]

#ensure all character vars are factors
ecom_train = ecom_train |> mutate(across(where(is.character), as.factor))
ecom_test = ecom_test |> mutate(across(where(is.character), as.factor))

#base tree
ecom_tree = rpart(as.factor(Churn) ~ ., data = ecom_train, method = "class")
rpart.plot(ecom_tree)

pred <- predict(ecom_tree, newdata = ecom_test, type = "class")

results = table(pred, ecom_test$Churn)
tree_sens = results["Churn", "Churn"] / sum(results[, "Churn"])
tree_spec = results["Not Churn", "Not Churn"] / sum(results[, "Not Churn"])

tree_error = 1 - sum(diag(results))/sum(results)

```

```

set.seed(7)
plotcp(ecom_tree)
ecom_tree$cptable

#find highest cp where xerror <= min(xerror) + xstd
min = which.min(ecom_tree$cptable[, "xerror"])
eligible = ecom_tree$cptable[ecom_tree$cptable[, "xerror"] <=
                           ecom_tree$cptable[min, "xerror"]
                           + ecom_tree$cptable[min, "xstd"], ]
best_cp = max(eligible[, "CP"])

ecom_tree_pruned = prune(ecom_tree, cp = best_cp)
rpart.plot(ecom_tree_pruned)

pred <- predict(ecom_tree_pruned, newdata = ecom_test, type = "class")

results = table(pred, ecom_test$Churn)
tree_pruned_sens = results["Churn", "Churn"] / sum(results[, "Churn"])
tree_pruned_spec = results["Not Churn", "Not Churn"] / sum(results[, "Not Churn"])

```

```

pruning_error = 1 - sum(diag(results))/sum(results)
library(randomForest)
set.seed(7)
ecom_bagging <- randomForest(as.factor(Churn) ~ ., data = ecom_train,
                             mtry = 18, importance = TRUE) #set mtry = p here

plot(ecom_bagging, col = 1:ncol(ecom_bagging$err.rate),
     main="Bagging OOB Error vs #Trees")
legend("topright", legend = colnames(ecom_bagging$err.rate),
     col = 1:ncol(ecom_bagging$err.rate), lty = 1,
     )

pred <- predict(ecom_bagging, newdata = ecom_test, type = "class")

results = table(pred, ecom_test$Churn)

bagging_error = 1 - sum(diag(results))/sum(results)
bagging_sens = results["Churn", "Churn"] / sum(results[, "Churn"])
bagging_spec = results["Not Churn", "Not Churn"] / sum(results[, "Not Churn"])

varImpPlot(ecom_bagging, main="Variable Importance: Bagging")
set.seed(7)
ecom_randomForest = randomForest(as.factor(Churn) ~ ., data = ecom_train,
                                 mtry = floor(sqrt(18)), #smaller mtry value
                                 importance = TRUE)

pred <- predict(ecom_randomForest, newdata = ecom_test, type = "class")

results = table(pred, ecom_test$Churn)

rf_error = 1 - sum(diag(results))/sum(results)
rf_sens = results["Churn", "Churn"] / sum(results[, "Churn"])
rf_spec = results["Not Churn", "Not Churn"] / sum(results[, "Not Churn"])

varImpPlot(ecom_randomForest, main="Variable Importance: Random Forest")
set.seed(7)

#arbitrary set of tuning params
tuning_par <- expand.grid("nodesize" = c(3, 7, 11, 15, 19),
                         "mtry" = c(7, 9, 11, 14, 18))
OOB_error <- rep(NA, nrow(tuning_par))

#loop through entire grid of pairs
for (k in seq_along(OOB_error)) {
  randomforest_fit <- randomForest(as.factor(Churn) ~ ., ntree = 500,
                                   nodesize = tuning_par$nodesize[k],
                                   mtry = tuning_par$mtry[k],
                                   data=ecom_train)
  OOB_error[k] <- randomforest_fit$err.rate[500, "OOB"]
}

```

```

results = cbind(tuning_par, OOB_error)

best = results[which.min(results$OOB_error),] #pick lowest OOB error combo

#best pairing model
set.seed(7)
randomforest_final <- randomForest(as.factor(Churn)~., ntree = 500,
                                   data=ecom_train,
                                   nodesize = best$nodesize, mtry = best$mtry,
                                   importance = TRUE)

varImpPlot(randomforest_final, main = "Variable Importance: Tuned Random Forest")

pred <- predict(randomforest_final, newdata = ecom_test, type = "class")

results <- table(pred, ecom_test$Churn)
tuned_sens = results["Churn", "Churn"] / sum(results[, "Churn"])
tuned_spec = results["Not Churn", "Not Churn"] / sum(results[, "Not Churn"])

rf_tuned_error = 1 - sum(diag(results)) / sum(results)

final_comp = data.frame(
  Model = c("Simple Classification Tree", "Pruned Classification Tree",
            "Bagging", "Random Forest", "Tuned Random Forest"),
  Test_Error = c(tree_error, pruning_error, bagging_error,
                 rf_error, rf_tuned_error),
  Sensitivity = c(tree_sens, tree_pruned_sens, bagging_sens, rf_sens, tuned_sens),
  Specificity = c(tree_spec, tree_pruned_spec, bagging_spec, rf_spec, tuned_spec)
)

final_comp

```